

راهنمای خواندن کد ChabokPro

یک جزوه‌ی قدم به قدم برای برنامه نویسی جونیور: یک request واقعی را از لحظه ورود به Laravel تا middlewareها، controller، FormRequest، handler، repository، transaction، audit، outbox و response دنبال می‌کنیم.

Endpoint: POST /api/v1/consignments	سناریوی اصلی: ساخت مرسوله در ماژول Consignment
تمرکز: خود کد، لایه‌ها، ورودی و خروجی هر مرحله	ماژول‌ها: Foundation, Identity, Authorization, Consignment

این فایل برای این نوشته شده که وقتی پروژه را تحویل گرفتی، فقط اسم کلاس‌ها را نبینی؛ بفهمی هر کلاس چرا اینجاست، چه ورودی می‌گیرد، چه خروجی می‌دهد و در مسیر یک request چه نقشی دارد.

بخش	چه چیزی یاد می‌گیری؟
request تستی	با چه header و body ای جلو می‌رویم.
لایه‌ها	فرق Presentation, Application, Domain و Infrastructure در همین مثال.
middlewareها	.idempotent و CorrelationId, NodeContext, access.auth, node.access, password.changed
Presentation	route, controller, FormRequest و normalize کردن ورودی.
Application	.transaction و command, handler, access guard, policy, pricing
Infrastructure	.outbox و provider binding, DB, audit, repository واقعی،
دیباگ	اگر 401, 403, 422 یا 500 دیدی از کجا شروع کنی.

1. request تستی که دنبال می‌کنیم

request ما برای ساخت مرسوله است. کاربر قبلا login کرده، access token دارد، یک node عملیاتی انتخاب کرده و یک quote قیمت را پذیرفته است.

```
POST /api/v1/consignments
Authorization: Bearer <access-token>
X-Node-Id: 018f77b6-7c5e-7c39-b4e2-aaaaaaaaaaaaa
X-Correlation-ID: 018f77b6-7c5e-7c39-b4e2-bbbbbbbbbbbbbb
Idempotency-Key: create-consignment-0001
```

body ساده شده:

```
{
  "sender": {
    "contact_name": "علی رضایی",
    "mobile": "09120000000",
    "address_text": "تهران، خیابان مثال",
    "city_id": "018f77b6-7c5e-7c39-b4e2-111111111111"
  },
  "receiver": {
    "contact_name": "سارا محمدی",
    "mobile": "09350000000",
    "address_text": "مشهد، خیابان نمونه",
    "city_id": "018f77b6-7c5e-7c39-b4e2-222222222222"
  },
  "service_type_id": "018f77b6-7c5e-7c39-b4e2-333333333333",
  "shipping_method_id": "018f77b6-7c5e-7c39-b4e2-444444444444",
  "weight_kg": 2.5,
  "declared_value_amount": 1000000,
  "insurance_enabled": true,
  "cod_enabled": false,
  "payer": "SENDER",
  "payment_method": "CASH",
  "parcels": [{"content_description": "کتاب", "weight_kg": 2.5}],
  "accepted_quote": {
    "quote_id": "018f77b6-7c5e-7c39-b4e2-555555555555",
    "quote_version": 1,
    "option_id": "018f77b6-7c5e-7c39-b4e2-666666666666"
  }
}
```

UUIDها و داده‌ها نمونه‌اند. هدف این است که مسیر کد را بفهمیم، نه اینکه این payload حتما بدون seed و دیتابیس مناسب اجرا شود.

2. نقشه‌ی کل مسیر

HTTP Request

```
-> Laravel bootstrap/app.php
-> Global API middleware
    -> CorrelationId
    -> RequireSecureTransport
    -> CredentialedCors
    -> NodeContext
-> Route middleware
    -> access.auth
    -> node.access
    -> password.changed
    -> idempotent:consignments.create
-> Consignment route
-> ConsignmentController@store
-> CreateConsignmentRequest
-> CreateConsignmentCommand
-> CreateConsignmentHandler
-> ConsignmentRepository / Pricing / Audit / Outbox
-> ApiResponse::success(..., 201)
```

هر مرحله یا request را آماده می‌کند، یا اجازه‌ی عبور می‌دهد، یا عملیات اصلی را انجام می‌دهد، یا پاسخ استاندارد می‌سازد.

3. لایه‌ها در همین سناریو

لایه	در این request چه فایل‌هایی دارد؟	مسئولیت دقیق
Presentation	Routes/api.php ConsignmentController.php CreateConsignmentRequest.php	ورود request، اعتبارسنجی فرم، ساخت command و تبدیل خروجی به response.
Application	CreateConsignmentCommand.php CreateConsignmentHandler.php ConsignmentAccessGuard.php ConsignmentRepository.php	اجرای use case و تعریف contractهایی که برای کار لازم است.
Domain	ConsignmentPolicy.php	قانون‌های تجاری مرسومه، مثل سازگاری پرداخت، بیمه، وزن و حالت pilot.
Infrastructure	EloquentConsignmentRepository.php ConsignmentServiceProvider.php RouteServiceProvider.php	اتصال به دیتابیس، provider، Redis، ها و implementation واقعی contractها.
Foundation	Middleware/* ApiResponse.php TransactionManager	رفتارهای مشترک کل پروژه: auth pipeline, response envelope, transaction, uuid, clock.

یک قانون ساده: اگر با HTTP طرف است احتمالاً Presentation است؛ اگر تصمیم use case است Application است؛ اگر قانون کسب‌وکار ناب است Domain است؛ اگر DB/Redis/Provider است Infrastructure است.

4. قبل از هر چیز: ماژول چطور route خودش را وارد Laravel می‌کند؟

ماژول Consignment یک ServiceProvider دارد. وقتی ماژول بالا می‌آید، migrationهای خودش را load می‌کند و RouteServiceProvider خودش را register می‌کند.

Modules/Consignment/src/Infrastructure/Providers/ConsignmentServiceProvider.php

```
public function boot(): void
{
    $this->loadMigrationsFrom(dirname(__DIR__, 3) . '/database/migrations');
    $this->app->register(RouteServiceProvider::class);
}
```

Modules/Consignment/src/Infrastructure/Providers/RouteServiceProvider.php

```
public function boot(): void
{
    $this->loadRoutesFrom(dirname(__DIR__, 2) . '/Presentation/Routes/api.php');
}
```

نتیجه: routeهای Consignment لازم نیست در فایل route اصلی Laravel نوشته شوند. خود ماژول routeهایش را load می‌کند.

5. bootstrap/app.php: ورودی همه‌ی APIها

bootstrap/app.php

```
$middleware->api(prepend: [
    CorrelationId::class,
    RequireSecureTransport::class,
    CredentialedCors::class,
    NodeContext::class,
]);

$middleware->alias([
    'access.auth' => AuthenticateAccessToken::class,
    'idempotent' => IdempotentCommand::class,
    'origin.exact' => RequireExactOrigin::class,
    'password.changed' => RequirePasswordChangeCompleted::class,
    'node.access' => ValidateNodeAccess::class,
]);
```

بخش api (prepend) یعنی این middlewareها قبل از middlewareهای معمول API اجرا می‌شوند. aliasها هم نام کوتاه middlewareهایی هستند که داخل route می‌بینی.

6. CorrelationId: نخ پیگیری request

Modules/Foundation/src/Presentation/Http/Middleware/CorrelationId.php

```

$provided = trim((string) $request->headers->get('X-Correlation-ID', ''));
$correlationId = $provided !== '' ? $provided : (string) Str::uuid();
$request->attributes->set('correlation_id', $correlationId);

Log::shareContext([
    'correlation_id' => $correlationId,
    'request_method' => $request->method(),
    'request_path' => '/' . $request->path(),
]);

if ($provided !== '' && !Str::isUuid($provided)) {
    $response = ApiResponder::error(..., 422, ['X-Correlation-ID' => [...]]);
}

```

در request تستی ما X-Correlation-ID وجود دارد. اگر UUID معتبر باشد، در request attribute ذخیره می‌شود و همگی log ها و response ها همان را حمل می‌کنند.

برای دیباگ production، این شناسه خیلی مهم است. هر خطا یا audit یا outbox مرتبط با همین request باید با همین correlation id قابل پیدا کردن باشد.

7. NodeContext: خواندن X-Node-Id

Modules/Foundation/src/Presentation/Http/Middleware/NodeContext.php

```
$nodeId = trim((string) $request->headers->get('X-Node-Id', ''));

if ($nodeId !== '' && !Str::isUuid($nodeId)) {
    throw new ApiException(
        ApiErrorCode::ValidationError,
        422,
        'The request is invalid.',
        ['X-Node-Id' => ['The X-Node-Id field must be a UUID.']]
    );
}

$request->attributes->set('node_id', $nodeId !== '' ? $nodeId : null);
```

این middleware هنوز نمی‌گوید کاربر اجازه دارد یا نه با node کار کند. فقط header را validate و ذخیره می‌کند. چک permission در middleware بعدی انجام می‌شود.

8. access.auth: تبدیل token به principal

Modules/Foundation/src/Presentation/Http/Middleware/AuthenticateAccessToken.php

```
$authorization = (string) $request->headers->get('Authorization', '');

if (!preg_match('/^Bearer\s+(.+)/i', $authorization, $matches)) {
    throw new ApiException(ApiErrorCode::AuthenticationRequired, 401, 'Authentication required.');
```

```
}

$principal = $this->sessions->validate($this->tokens->decode($matches[1]));
$request->attributes->set('principal', $principal);
$request->attributes->set('tenant_id', $principal->hqId);
```

خروجی مهم این middleware شیء AuthenticatedPrincipal است:

Modules/Foundation/src/Domain/AuthenticatedPrincipal.php

```
final readonly class AuthenticatedPrincipal
{
    public function __construct(
        public string $userId,
        public string $sessionId,
        public ?string $hqId,
        public bool $mustChangePassword,
    ) {}
}
```

بعد از این مرحله، controller و handler لازم نیست token را بشناسند. فقط با principal کار می‌کنند.

9. node.access: اجازه کار با node

Modules/Foundation/src/Presentation/Http/Middleware/ValidateNodeAccess.php

```
$nodeId = $request->attributes->get('node_id');
$principal = $request->attributes->get('principal');

if (is_string($nodeId) && $principal instanceof AuthenticatedPrincipal) {
    $this->validator->assertAccessible($principal, $nodeId);
    $this->audit->write(
        $principal->hqId,
        $principal->userId,
        'NODE_CONTEXT_SELECTED',
        'NODE',
        $nodeId,
        (string) $request->attributes->get('correlation_id'),
        sourceClient: 'BRANCH_PANEL'
    );
}
```

اینجا دو کار انجام می‌شود: اول دسترسی به node چک می‌شود، بعد یک audit سبک نوشته می‌شود که user این node را انتخاب کرده است.

چک واقعی دسترسی از طریق contract انجام می‌شود. یعنی Foundation فقط می‌گوید «یک NodeAccessValidator لازم دارم»، اما پیاده‌سازی واقعی از Authorization می‌آید.

10. password.changed: اگر کاربر باید رمز عوض کند

Modules/Foundation/src/Presentation/Http/Middleware/RequirePasswordChangeCompleted.php

```
$principal = $request->attributes->get('principal');

if (
    $principal instanceof AuthenticatedPrincipal
    && $principal->mustChangePassword
    && !in_array($request->route()->getName(), config('chabok.forced_password_allowed_routes', []), true)
) {
    throw new ApiException(ApiErrorCode::PasswordChangeRequired, 403, 'Password change is required.');
```

اگر user باید رمز عوض کند، به route‌های معمولی مثل ساخت مرسوله دسترسی ندارد. فقط route‌هایی که در config مجاز شده‌اند باز می‌مانند.

11. idempotent: محافظت عملیات create

Modules/Foundation/src/Presentation/Http/Middleware/IdempotentCommand.php

```
$key = (string) $request->header('Idempotency-Key', '');

if (strlen($key) < 16 || strlen($key) > 128) {
    throw new ApiException(ApiErrorCode::ValidationError, 422, ...);
}

$fingerprint = hash(
    'sha256',
    $request->method() . '|' . $request->path() . '|' . $request->getContent()
);
```

Idempotency-Key کمک می‌کند اگر کاربر دوبار کلیک کرد یا frontend request را retry کرد، یک عملیات create دوبار اجرا نشود.

حالت	نتیجه
key کوتاه‌تر از 16 کاراکتر	خطای validation 422
همان key و همان body	پاسخ قبلی می‌تواند replay شود
همان key ولی body متفاوت	باید conflict یا خطای مرتبط با idempotency بدهد

12. route مازول Consignment

Modules/Consignment/src/Presentation/Routes/api.php

```
Route::prefix('api/v1/consignments')
->middleware(['api', 'access.auth', 'node.access', 'password.changed'])
->group(function (): void {
    Route::post('/', [ConsignmentController::class, 'store'])
        ->middleware('idempotent:consignments.create')
        ->name('consignments.store');
});
```

request ما دقیقاً با این route match می‌شود. پس بعد از middleware ها وارد متد store در ConsignmentController می‌شود.

اگر خواستی هر feature را بفهمی، اول route را پیدا کن. route به تو می‌گوید controller چیست، middleware ها چیستند و اسم route چیست.

13. controller: نازک و قابل خواندن

Modules/Consignment/src/Presentation/Http/Controllers/ConsignmentController.php

```
public function store(CreateConsignmentRequest $request): JsonResponse
{
    $input = $request->validated();
    $input = $request->normalizePilotCreate($input);

    return ApiResponder::success(
        $request,
        $this->create->handle(new CreateConsignmentCommand(
            $this->principal($request),
            $this->nodeId($request),
            $input,
            $this->correlationId($request)
        ))->data,
        status: 201
    );
}
```

controller اینجا فقط نقش مترجم را دارد:

1. request خام HTTP را به داده validate شده تبدیل می‌کند.
2. چند مقدار context مثل principal, nodeId و correlationId را از request attribute برمی‌دارد.
3. یک command می‌سازد.
4. handler را صدا می‌زند.
5. خروجی را با status 201 برمی‌گرداند.

14. CreateConsignmentRequest: محافظ شکل ورودی

Modules/Consignment/src/Presentation/Http/Requests/CreateConsignmentRequest.php

```
final class CreateConsignmentRequest extends ConsignmentInputRequest
{
    public function rules(): array
    {
        return [...$this->draftRules(), ...$this->acceptedQuoteRules()];
    }

    protected function prepareForValidation(): void
    {
        StrictPayload::assertOnly($this, [...self::DRAFT_FIELDS, 'accepted_quote']);
        $this->assertNestedPayload($this, true);
    }
}
```

دو نکته مهم:

- `rules()` می‌گوید چه فیلدهایی از نظر `validation` قابل قبول اند.
 - `prepareForValidation()` قبل از `validation` فیلدهای اضافه را رد می‌کند.
- پس اگر `frontend` اشتباهی فیلد ناشناس بفرستد، سیستم ساکت آن را `ignore` نمی‌کند؛ `request` را `invalid` می‌کند.

15. rules اصلی body

Modules/Consignment/src/Presentation/Http/Requests/ConsignmentInputRequest.php

```
'service_type_id' => ['required_without:service_offering_id', 'nullable', 'uuid'],
'shipping_method_id' => ['required_without:service_offering_id', 'nullable', 'uuid'],
'weight_kg' => ['required', 'numeric', 'gt:0'],
'declared_value_amount' => ['required', 'integer', 'min:0'],
'insurance_enabled' => ['required', 'boolean'],
'cod_enabled' => ['required', 'boolean'],
'payer' => ['required', 'in:SENDER,RECEIVER,VENDOR'],
'payment_method' => ['required', 'in:CASH,CREDIT,COD'],
'parcels' => ['required', 'array', 'min:1', 'max:100'],
```

فیلد	چرا مهم است؟
weight_kg	وزن باید عدد و بزرگتر از صفر باشد.
declared_value_amount	ارزش اظهار شده برای بیمه و قیمت‌گذاری استفاده می‌شود.
payer	مشخص می‌کند پرداخت با فرستنده، گیرنده یا vendor است.
parcels	مرسوله بدون بسته معنی ندارد؛ حداقل یک parcel لازم است.
accepted_quote	یعنی client قبل از create قیمت را گرفته و یکی از گزینه‌ها را پذیرفته است.

16. normalizePilotCreate: قانون مخصوص pilot

ConsignmentInputRequest.php

```

public function normalizePilotCreate(array $input): array
{
    if (($input['insurance_enabled'] ?? null) !== true) {
        throw ValidationException::withMessages(['insurance_enabled' => [...]]);
    }
    if (!in_array($input['payer'] ?? null, ['SENDER', 'RECEIVER'], true)) {
        throw ValidationException::withMessages(['payer' => [...]]);
    }
    if (!in_array($input['payment_method'] ?? null, ['CASH', 'CREDIT'], true)) {
        throw ValidationException::withMessages(['payment_method' => [...]]);
    }

    $input['insurance_enabled'] = true;
    $input['insurance_value_amount'] = (int) $input['declared_value_amount'];
    return $input;
}

```

در pilot، بیمه اجباری است، payer فقط فرستنده یا گیرنده است، و payment method فقط cash یا credit است. همچنین مبلغ بیمه برابر declared value می‌شود.

17. Command: بسته‌ی داده برای use case

Modules/Consignment/src/Application/UseCases/CreateConsignment/CreateConsignmentCommand.php

```
final readonly class CreateConsignmentCommand
{
    public function __construct(
        public AuthenticatedPrincipal $actor,
        public string $nodeId,
        public array $input,
        public string $correlationId,
    ) {}
}
```

command در این پروژه بیشتر یک DTO ساده است. هدفش این است که handler پارامترهای لازم را مرتب و type دار دریافت کند.

property	از کجا آمده؟	کجا مصرف می‌شود؟
actor	middleware access.auth	initiator_id و permission, audit, tenant
nodeId	header X-Node-Id	pickup_node_id, custody, scope check
input	FormRequest validated	pricing و consignment, parcel ساخت
correlationId	CorrelationId middleware	audit, outbox, response, log

18. Handler constructor: وابستگی‌ها به ما چه می‌گویند؟

CreateConsignmentHandler.php

```
public function __construct(
    private ConsignmentAccessGuard $consignmentAccessGuard,
    private GeographyResolver $geography,
    private ConsignmentPolicy $policy,
    private ConsignmentRepository $consignments,
    private PricingService $pricing,
    private ConsignmentDraft $consignmentDraft,
    private TransactionManager $transactions,
    private IdentifierGenerator $identifiers,
    private ConsignmentNumberAllocator $numbers,
    private Clock $clock,
    private ConsignmentHistoryWriter $consignmentHistoryWriter,
    private ConsignmentPricingWriter $consignmentPricingWriter,
    private AuditWriter $audit,
    private OutboxWriter $outbox,
    private GetConsignmentHandler $getConsignment,
) {}
```

قبل از خواندن بدنه handler, constructor به تو می‌گوید این use case با چه چیزهایی کار دارد: access, geography, policy, pricing, transaction, شماره مرسوله, تاریخ, audit, outbox و خواندن خروجی نهایی.

19. Handler مرحله اول: access و آماده سازی input

CreateConsignmentHandler.php

```
$context = $this->consignmentAccessGuard
    ->assertAccess($actor, $nodeId, 'consignment.create');

$acceptedInput = (array) $input['accepted_quote'];
unset($input['accepted_quote']);

foreach (['sender', 'receiver'] as $party) {
    $input[$party] = $this->geography
        ->canonicalizeContact((array) $input[$party], true);
}

$this->policy->assertCommercialConsistency($input);
$this->policy->assertPilotCreate($input);
```

ترتیب این بخش مهم است:

1. اول دسترسی چک می شود؛ اگر کاربر اجازه ندارد، جلوتر نمی رویم.
2. quote پذیرفته شده جدا می شود تا payload اصلی مرسوله تمیز بماند.
3. آدرس فرستنده و گیرنده با geography canonical می شود.
4. قانون های دامنه ای چک می شوند.

20. ConsignmentAccessGuard: چهار قفل دسترسی

Modules/Consignment/src/Application/Services/ConsignmentAccessGuard.php

```
if ($actor->hqId === null) {
    throw new ApiException(ApiErrorCode::TenantAccessDenied, 403, 'Access denied.');
```

```
}

$context = $this->authorization->resolve($actor);

$entitled = array_filter(
    $context['module_entitlements'],
    fn(array $item): bool =>
        $item['module_code'] === 'Consignment' && $item['status'] === 'ENABLED'
) !== [];

if (!$entitled) throw new ApiException(ApiErrorCode::EntitlementDisabled, 403, ...);
if (!in_array($permission, $context['permissions'], true)) throw new ApiException(...);
if (!in_array($nodeId, $this->scopedAccess->nodes($context, $permission), true)) throw new
ApiException(...);
```

معنی	قفل
کاربر باید به یک tenant/hq وصل باشد.	tenant
ماژول Consignment برای tenant فعال باشد.	entitlement
کاربر permission مثل consignment.create داشته باشد.	permission
permission کاربر باید همین node را پوشش دهد.	scope

21. Pricing: پذیرفتن quote

CreateConsignmentHandler.php

```
$accepted = $this->pricing->accept(
    $actor,
    $nodeId,
    'CREATE',
    $input,
    $acceptedInput
);

if (($accepted['provider_code'] ?? null) === 'INTERNAL') {
    $input['service_type_id'] = $accepted['service_type_id'];
    $input['shipping_method_id'] = $accepted['shipping_method_id'];
    $input['service_offering_id'] = $accepted['service_offering_id'];
    $input['service_offering_version_id'] = $accepted['service_offering_version_id'];
    $input['selected_option_version_ids'] = $accepted['selected_option_version_ids'];
    $input = $this->consignmentDraft
        ->withAcceptedCommitment($input, $accepted['commitment'] ?? null);
}
```

اینجا handler مطمئن می‌شود quote‌ای که client فرستاده با همین input فعلی سازگار است. اگر pricing داخلی باشد، اطلاعات service و commitment از quote پذیرفته شده وارد input می‌شود.

22. TransactionManager: مرز اتمیک عملیات

Modules/Foundation/src/Infrastructure/Persistence/LaravelTransactionManager.php

```
final class LaravelTransactionManager implements TransactionManager
{
    public function __construct(private readonly int $attempts = 3) {}

    public function run(callable $callback): mixed
    {
        return DB::transaction($callback, $this->attempts);
    }
}
```

در ساخت مرسوله چند جدول تغییر می‌کنند. اگر وسط کار خطا بدهد، نباید فقط بخشی از داده ذخیره شود. برای همین عملیات‌های مهم داخل transaction هستند.

23. داخل transaction: ساخت رکورد مرسوله

CreateConsignmentHandler.php

```

$consignmentId = $this->transactions->run(function () use (...) {
    $id = $this->identifiers->uuid();
    $allocation = $this->numbers->next((string) $actor->hqId);
    $number = $allocation['consignment_number'];
    $now = $this->clock->now();
    $parcels = (array) $input['parcels'];

    $this->consignments->insertConsignment([
        'consignment_id' => $id,
        'hq_id' => $actor->hqId,
        'consignment_number' => $number,
        'initiator_id' => $actor->userId,
        'pickup_node_id' => $nodeId,
        'delivery_node_id' => $input['delivery_node_id'] ?? null,
        'current_status' => 'CFM',
        'aggregate_mode' => 'FULL',
        'parcel_status_counts' => json_encode(['CFM' => count($parcels)], JSON_THROW_ON_ERROR),
        'version' => 1,
        'created_at' => $now,
        'updated_at' => $now,
    ]);
});

```

در این قسمت، uuid ساخته می‌شود، شماره مرسوله تخصیص داده می‌شود، زمان فعلی گرفته می‌شود، و رکورد اصلی consignment ذخیره می‌شود.

24. ساخت parcel ها و custody

CreateConsignmentHandler.php

```
foreach (array_values($parcels) as $index => $parcelInput) {
    $parcelId = $this->identifiers->uuid();

    $this->consignments->insertParcel([
        'parcel_id' => $parcelId,
        'hq_id' => $actor->hqId,
        'consignment_id' => $id,
        'parcel_number' => sprintf('%s-%02d', $number, $index + 1),
        'current_status' => 'CFM',
        'current_node_id' => $nodeId,
        'current_custody_type' => 'NODE',
        'current_custodian_id' => $nodeId,
    ]);

    $this->consignments->appendCustodyEvent([
        'from_node_id' => null,
        'to_node_id' => $nodeId,
        'command_name' => 'CONSIGNMENT_CREATED',
        'initiator_id' => $actor->userId,
    ]);
}
```

برای هر item در آرایه parcels یک parcel ساخته می‌شود. چون مرسوله تازه ایجاد شده، بسته‌ها در همان node انتخاب شده قرار می‌گیرند.

25. status history و pricing version

CreateConsignmentHandler.php

```
$this->consignmentHistoryWriter->insertStatusEvent(
    (string) $actor->hqId,
    $id,
    $parcelId,
    $actor->userId,
    $nodeId,
    null,
    'CFM',
    'CONSIGNMENT_CONFIRMED',
    $correlationId
);

$pricingVersionId = $this->consignmentPricingWriter->persistPricing(
    (string) $actor->hqId,
    $id,
    1,
    $actor->userId,
    $accepted
);
```

status event می‌گوید مرسوله/بسته با وضعیت CFM تایید شده است. pricing writer هم قیمت پذیرفته شده را به عنوان نسخه قیمت مرسوله ذخیره می‌کند.

CreateConsignmentHandler.php

```
$this->audit->write(
    $actor->hqId,
    $actor->userId,
    'CONSIGNMENT_CREATED',
    'CONSIGNMENT',
    $id,
    $correlationId,
    after: ['version' => 1, 'status' => 'CFM', 'parcel_count' => count($parcels)],
    sourceClient: 'BRANCH_PANEL'
);

$this->outbox->write(
    $actor->hqId,
    'CONSIGNMENT',
    $id,
    'consignment.created',
    $correlationId,
    ['consignment_id' => $id, 'version' => '1', 'status' => 'CFM']
);
```

مفهوم	برای چیست؟
Audit	ردگیری انسانی: چه کسی چه کاری انجام داد؟
Outbox	ردگیری سیستمی/event-driven: چه رخدادی باید بعداً منتشر شود؟

27. Repository contract: handler دیتابیس را مستقیم نمی شناسد

Modules/Consignment/src/Application/Repositories/ConsignmentRepository.php

```
interface ConsignmentRepository
{
    public function activeNodeExists(string $nodeId, string $hqId): bool;
    public function insertConsignment(array $attributes): void;
    public function insertParcel(array $attributes): void;
    public function appendCustodyEvent(array $attributes): void;
    public function appendStatusEvent(array $attributes): void;
    public function nextCustodySequence(string $consignmentId): int;
    public function nextStatusSequence(string $consignmentId): int;
}
```

Application layer فقط contract را می شناسد. یعنی handler می گوید «من لازم دارم consignment ذخیره کنم»، اما نمی گوید با MySQL، Eloquent یا DB facade چطور این کار انجام شود.

28. Repository implementation: query واقعی

Modules/Consignment/src/Infrastructure/Repositories/EloquentConsignmentRepository.php

```
final class EloquentConsignmentRepository implements ConsignmentRepository
{
    public function activeNodeExists(string $nodeId, string $hqId): bool
    {
        return DB::table('nodes')
            ->where([
                'hq_id' => $hqId,
                'node_id' => $nodeId,
                'status' => 'ACTIVE',
            ])
            ->exists();
    }
}
```

اینجا Infrastructure layer است، پس دیدن DB::table و model ها طبیعی است. اگر لازم باشد دیتابیس عوض شود، ideally handler نباید تغییر کند؛ implementation عوض می شود.

29. ServiceProvider: اتصال contract به implementation

Modules/Consignment/src/Infrastructure/Providers/ConsignmentServiceProvider.php

```
public function register(): void
{
    $this->app->singleton(
        \Modules\Consignment\Application\Repositories\ConsignmentRepository::class,
        \Modules\Consignment\Infrastructure\Repositories\EloquentConsignmentRepository::class
    );

    $this->app->singleton(
        PricingQuoteProvider::class,
        static fn($app) =>
            config('chabok.pricing.provider', 'legacy') === 'internal'
                ? $app->make(InternalPricingAdapter::class)
                : $app->make(LegacyCorePricingAdapter::class)
    );
}
```

وقتی Laravel می‌خواهد CreateConsignmentHandler را بسازد، constructor آن یک ConsignmentRepository می‌خواهد. این provider می‌گوید implementation واقعی آن EloquentConsignmentRepository است.

CreateConsignmentHandler.php

```
$this->pricing->consume((string) $accepted['quote_id']);

return $this->getConsignment->handle(
    new GetConsignmentCommand($actor, $nodeId, $consignmentId)
)->data;
```

بعد از quote، transaction، مصرف می‌شود و handler با GetConsignmentHandler نسخه نهایی مرسوله را می‌خواند. این بهتر از این است که خودش دستی response بسازد، چون response از همان مسیر read model پروژه می‌آید.

Modules/Foundation/src/Presentation/Http/ApiResponse.php

```
public static function success(Request $request, mixed $data, array $meta = [], int $status = 200):
    JsonResponse
{
    $correlationId = self::correlationId($request);
    return response()->json([
        'data' => $data,
        'meta' => (object) $meta,
        'correlation_id' => $correlationId,
    ], $status)->header('X-Correlation-ID', $correlationId);
}
```

پس response موفق معمولاً این شکل را دارد:

```
HTTP 201
{
  "data": {
    "consignment_id": "...",
    "consignment_number": "...",
    "current_status": "CFM",
    "parcels": [...]
  },
  "meta": {},
  "correlation_id": "018f77b6-7c5e-7c39-b4e2-bbbbbbbbbbbb"
}
```

31. اگر خطا بخورد، کجا تبدیل به response می‌شود؟

Modules/Foundation/src/Presentation/Http/ApiExceptionRenderer.php

```

return match (true) {
  $throwable instanceof ApiException =>
    ApiResponse::error($request, $throwable->errorCode, $throwable->getMessage(), ...),

  $throwable instanceof ValidationException =>
    ApiResponse::error($request, ApiErrorCode::ValidationError, 'The request is invalid.', 422, ...),

  $throwable instanceof AuthenticationException =>
    ApiResponse::error($request, ApiErrorCode::AuthenticationRequired, 'Authentication required.',
401),

  default =>
    ApiResponse::error($request, ApiErrorCode::InternalServerError, 'An unexpected error occurred.',
500),
};

```

احتمالا از کجا آمده؟	Status
Authorization header نیست، token خراب است، session معتبر نیست.	401
permission, scope, tenant, entitlement یا password change مشکل دارد.	403
.FormRequest, StrictPayload, UUID header, business validation	422
تداخل دیتابیس یا conflict عملیاتی.	409
خطای پیش‌بینی نشده؛ correlation id را در log دنبال کن.	500

32. چک لیست دیباگ همین request

مشکل	اول کجا را ببین؟	بعد کجا؟
route پیدا نمی شود	Modules/Consignment/src/Presentation/Routes/api.php	RouteServiceProvider ماژول
401	AuthenticateAccessToken.php	AccessTokenService و SessionValidator
X-Node-Id خطا دارد	NodeContext.php	ValidateNodeAccess.php
access 403 denied	ConsignmentAccessGuard.php	AuthorizationService و scopeها
body invalid 422	CreateConsignmentRequest.php	ConsignmentInputRequest.php
مرسوله دوبار ساخته شده	IdempotentCommand.php	fingerprint و Idempotency-Key
شماره مرسوله مشکل دارد	ConsignmentNumberAllocator	number range repository
event منتشر نشده	OutboxWriter	publication_state و outbox worker

33. یک روش عملی برای خواندن featureهای دیگر

1. از route شروع کن؛ endpoint و middlewareها را پیدا کن.
2. controller را باز کن؛ ببین request class و handler کدام است.
3. FormRequest را بخوان؛ ورودی مجاز و خطاهای 422 را از آن بفهم.
4. Command را نگاه کن؛ بفهم handler چه داده هایی می گیرد.
5. Handler را مرحله ای بخوان؛ هر dependency در constructor یک سرخ است.
6. Guard و Policy را جدا بخوان؛ access و business rule معمولاً آنجاست.
7. Repository interface را بخوان؛ بفهم use case چه عملیات دیتابیزی لازم دارد.
8. Implementation را فقط وقتی بخوان که می خواهی query واقعی را بفهمی.
9. Provider را چک کن؛ ببین contractها به چه کلاس هایی وصل شده اند.
10. برای دیباگ runtime, correlation id را از response بگیر و در log/audit/outbox دنبال کن.

34. خلاصه نهایی مسیر request

```
POST /api/v1/consignments
-> bootstrap/app.php
  -> CorrelationId: ساخت/اعتبارسنجی correlation_id
  -> NodeContext: خواندن X-Node-Id
  -> access.auth: ساخت principal
  -> node.access: بررسی node
  -> password.changed: اجبار تغییر رمز
  -> idempotent: تکراری create جلوگیری از
-> Consignment Routes
-> ConsignmentController@store
-> CreateConsignmentRequest
  -> rules
  -> StrictPayload
  -> normalizePilotCreate
-> CreateConsignmentCommand
-> CreateConsignmentHandler
  -> ConsignmentAccessGuard
  -> GeographyResolver
  -> ConsignmentPolicy
  -> PricingService
  -> TransactionManager
  -> ConsignmentRepository
  -> AuditWriter
  -> OutboxWriter
-> GetConsignmentHandler
-> ApiResponder::success(status: 201)
```

اگر فقط یک چیز از این جزوه یادت بماند: این پروژه را از route به controller، از controller به request و command، از command به handler، و از handler به repository/provider دنبال کن.